

**Projet de compilation du langage MiniAlgo
Avec les Outils FLEX et BISON**

Objectif:

Réaliser un mini-compilateur pour le langage **MiniAlgo** en implémentant les différentes phases de compilation :

1. Analyse lexicale avec Flex
2. Analyse syntaxique et sémantique avec Bison
3. Gestion de la table des symboles
4. Génération de code intermédiaire (quadruplets)
5. Détection et affichage des erreurs

1. Structure du langage MiniAlgo

Un programme MiniAlgo a la structure suivante :

```
PROGRAM NomProgramme
DECL
  Déclarations
ENDDECL
BEGIN
  Instructions
END
```

2. Les éléments du langage

2.1 Commentaires

Deux types de commentaires :

```
// commentaire sur une ligne
```

ou

```
/* commentaire  
sur plusieurs lignes */
```

2.2 Déclarations

Variables simples

```
TYPE : liste_identificateurs ;
```

Exemple

```
INTEGER : A, B, C ;  
FLOAT : X ;
```

Tableaux

```
TYPE : IDF [taille] ;
```

Exemple

```
INTEGER : T[10] ;
```

Constantes

```
CONST IDF = valeur ;
```

Exemple

```
CONST PI = 3.14 ;  
CONST MAX = 100 ;
```

2.3 Types supportés

- INTEGER

- FLOAT

Contraintes :

- $\text{INTEGER} \in [-32768, 32767]$
- Les valeurs signées doivent être entre parenthèses

Exemple

```
(-5)
(+12)
(-2.7)
```

2.4 Identificateurs

Règles :

- Commence par une lettre
- Peut contenir lettres, chiffres et _
- Taille maximale : 8 caractères

Exemples valides

```
A
Somme
val_1
```

Exemples invalides

```
1A
abc-def
variable_tres_longue
```

3. Instructions

Toutes les instructions se terminent par ;

3.1 Affectation

```
IDF = expression ;
```

Exemple

```
A = B + 5 ;  
X = (A * 2) + 7 ;
```

3.2 Condition

```
IF (condition) {  
    instructions  
}  
ELSE {  
    instructions  
}
```

Exemple

```
IF (A > B) {  
    MAX = A ;  
}  
ELSE {  
    MAX = B ;  
}
```

3.3 Boucle FOR

```
FOR (i : debut : pas : fin) {  
    instructions  
}
```

Exemple

```
FOR (i : 0 : 1 : 10) {  
    A = A + i ;  
}
```

3.4 Boucle WHILE

```
WHILE (condition) {  
    instructions  
}
```

Exemple

```
WHILE (i < 10) {
```

```
i = i + 1 ;  
}
```

4. Opérateurs

Arithmétiques	+ - * /
Logiques	&& !
Comparaison	> < >= <= == !=

5. Travail demandé

Partie 1 : Analyse lexicale (Flex)

Définir les tokens suivants :

- Mots clés
- Identificateurs
- Constantes entières
- Constantes réelles
- Opérateurs
- Délimiteurs

Le programme doit :

- ignorer les commentaires
- remplir la table des symboles
- détecter les erreurs lexicales

Partie 2 : Analyse syntaxique (Bison)

Écrire la grammaire du langage MiniAlgo.

Le parser doit vérifier :

- structure du programme
- déclarations
- instructions valides

Partie 3 : Table des symboles

La table des symboles est construite durant la phase d'analyse lexicale. Elle regroupe toutes les variables et constantes déclarées par le programmeur, ainsi que les informations nécessaires au processus de compilation. Cette table est implémentée sous forme d'une table de hachage et mise à jour progressivement au fur et à mesure de l'avancement de la compilation.

Il est demandé de prévoir des procédures permettant la recherche et l'insertion d'éléments dans la table des symboles. Les variables structurées, notamment celles de type tableau, doivent également y être enregistrées.

Partie 4 : Code intermédiaire

Le code intermédiaire doit être généré sous forme de quadruplets.

Partie 5 : Gestion des erreurs

Votre compilateur doit détecter :

Erreur	Exemple
Erreurs lexicales	identificateur invalide
Erreurs syntaxiques	point-virgule manquant
Erreurs sémantiques	variable non déclarée incompatibilité de type double déclaration division par zéro tableau (taille invalide) Tableau (type d'élément invalide) Tableau (accès à de mauvais indices dans un tableau)

Format d'erreur :

Erreur Syntaxique : ligne X , colonne Y , élément Z

6. Optimisation du Code Intermédiaire

Appliquer des optimisations sur le code intermédiaire, telles que la réduction de code inutile, l'élimination du code mort, la simplification des expressions constantes, etc.

7. Exemple de programme valide

Le code machine doit être généré en assembleur 8086.

8. Exemple de programme valide

```
PROGRAM Test
DECL
INTEGER : A, B ;
FLOAT : X ;
CONST MAX = 10 ;
ENDDECL

BEGIN
A = 5 ;
B = 3 ;

IF (A > B) {
    WRITE(A);
}
ELSE {
    WRITE(B);
}

END
```